

Part 1 - Path finding

This part of the assignment focuses on pathfinding algorithms. The goal is to efficiently find a path between a source and a destination. To demonstrate this, we will use the mechanics of a Pac-Man game to explore different pathfinding strategies.

i) Understanding the Game

In this game, you will play as Pac-Man, and your objective is to find a path from your starting location to the enemy's location. You can move to neighboring cells—up, down, left, or right—but some locations will be inaccessible, meaning they cannot be included in your path. Keep in mind that diagonal cells do not count as neighbors.

We will use three different algorithms to find the path: BFS, DFS, and A* search.



ii) Understanding the Code File

For this part of the assignment you are given 6 coding files.

- main.py
- bfs.py
- dfs.py
- A_star.py

- requirement.txt
- utils.py

Before you start open the assignment folder and run the following command in your terminal
pip install -r requirements.txt

You will be interacting with bfs.py , dfs.py, A_star.py and in case you want to implement any extra function then in that case you can interact with utils.py too.

The purpose of main.py is to visualize the path on the grid, hence you are not allowed to edit the file.

You need to implement 4 functions in this part only. **Dfs**, **Bfs** , **heuristic** and **A_star**.

- **Inorder to see the visualization and the results please run the main.py file.**

iii) Assignment (To Do!)

As discussed above for part 1 you only need to implement 4 functions.

Bfs : (10 marks)

The function takes four arguments: the grid, the starting cell, the destination (end cell), and the size of the grid. For your convenience, the function and its arguments have already been defined.

Your task is to implement the Breadth-First Search (BFS) algorithm to find the shortest path and return two lists:

1. **Final Path: A list of tuples representing the coordinates of cells from the start to the destination.**
2. **All Explored Paths: A list of tuples representing the coordinates of every cell visited in the order they were explored.**

Each tuple should contain the coordinates of a cell, and the first tuple in both lists must be the starting cell, while the last tuple must be the destination. For the "All Explored Paths," ensure that every visited cell is included in the order they were traversed.

Dfs : (10 marks)

This function also takes the same 4 arguments as bfs and has been defined for your ease. Your task is to Implement DFS and Return the final path & all paths explored in order of traversal.

Similar to Bfs you are required to return the final path and all explored paths/cells in order of traversal in the same way as mentioned in bfs.

Heuristic:

This function calculates the Manhattan Distance between two points (cell1 and cell2) on the grid. The Manhattan Distance is used as a heuristic to estimate the remaining distance from the current cell to the destination (end point). It helps guide the algorithm in the right direction by providing an estimate of how far a cell is from the goal.

Purpose: The Manhattan Distance helps prioritize cells that are closer to the goal, improving the efficiency of the search algorithm.

Formula:

$$\text{Manhattan Distance} = |x_2 - x_1| + |y_2 - y_1|$$

Where:

(x1,y1) are the coordinates of cell1.

(x2,y2) are the coordinates of cell2.

A_star: (20 marks)

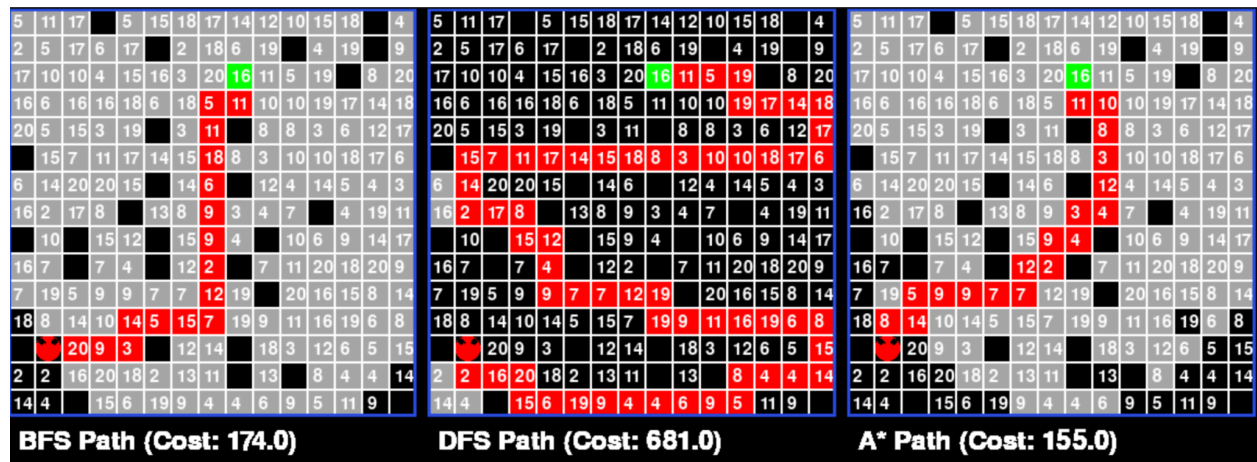
This function uses the same arguments as BFS and DFS: the grid, the starting cell, the destination (end cell), and the size of the grid. The purpose of this function is to find the most efficient path or the path with the lowest movement costs from the source to the destination.

Movement Cost: Movement cost is the actual cost incurred while moving from the start to the current cell. The cost depends on the values in the grid, where each cell may have a different movement cost (e.g., higher values mean it's more "expensive" to pass through that cell). The A* algorithm tracks this cost to ensure that the total path cost is minimized.

Purpose: The movement cost accumulates as the algorithm explores the grid, representing the actual cost to reach a cell from the start. The A* algorithm combines the actual movement cost with the heuristic (Manhattan Distance) to efficiently find the lowest-cost path.

Verification: To ensure the function works correctly, check the cost for A* during visualization. It should be less than the costs reported by BFS and DFS, reflecting its efficiency. Verify this

through multiple runs to confirm that A* consistently finds a path with the lowest movement cost.



Part 2 - Minimax

This part of the assignment focuses on the minimax algorithm which is used for decision making and can be applied to a 2 person game. To test your understanding and application of the concept, we will try and code the game **Connect4**.

i) Understanding the Game

The game is played on a 6x7 board where two players take turns dropping their coloured - or numbered in this case - pieces into the columns. The first player to get 4 of their pieces in a row (horizontally, vertically, or diagonally) wins the game. If the board fills up before either player wins, the game is a draw. An example of the board is given below:

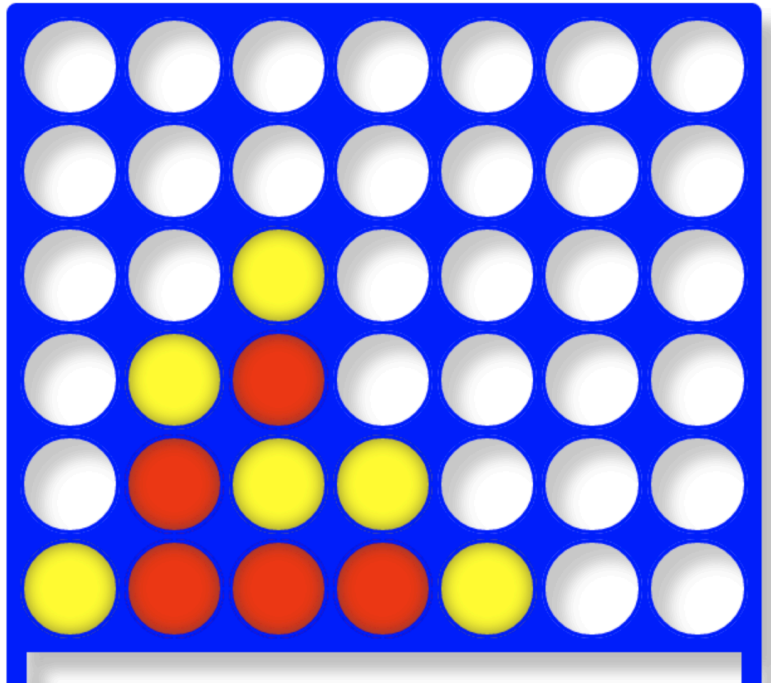


Fig: Connect 4 Board

You can try playing the game to get familiar with the rules: [Connect4](#)

ii) Understanding the Code File

This assignment is to be done on the Python Notebook provided. The Notebook has all relevant information to tackle the assignment along with foundation code for game mechanics (e.g. setting up board, managing turns, etc). You only need to worry about the Minimax implementation.

To start off, in the first code cell given, you can uncomment the first 2 lines to install the relevant Python libraries for this assignment (remember you must run the cells in the order given to prevent any errors of missing variables and undeclared functions). The following cells set up the game for you.

You only have to code in 2 functions - as indicated in the notebook - which include the 'winning_move' function which checks for the winning condition, and 'minimax' which implements the minimax algorithm.

iii) Assignment (To Do!)

As stated above, you only have to code 2 functions: 'winning_move' and 'minimax'.

A - winning-move (10 marks)

This function takes in 2 arguments: *board*, which is the current board state, and *piece* which indicates whose turn it is (1 for player, and 2 for computer). The function returns *True* if the current piece has won, or returns *False* if not. This function is called after every turn to check if the game has ended, before switching turns.

For this function, you need to check if in the given board state, the piece (or player) has met the win condition which is to get 4 pieces in a row (horizontally, vertically, or diagonally). This is a fairly easy function just to test basic programming.

B - minimax (30 marks)

This function takes in 5 arguments:

- *Board* - the current board state
- *Depth* - the number of levels or moves ahead that the algorithm explores in the game tree
- *maximizingPlayer* - indicating whether we are looking for maximum score or minimum

The function returns a tuple (*column*, *value*) which indicates the best column to drop a piece corresponding to the best move, and the heuristic value of the move.

Hint! - For this function make use of the *get_next_open_row*, *drop_piece*, and *score_position* functions.